

CS427 - Team 407 - Final Report

Final Performance:

Subset	Resolved Count	Resolved ID
verified-40	22	"django__django-10880", "django__django-11095", "django__django-11265", "django__django-11532", "django__django-11880", "django__django-13089", "django__django-13363", "django__django-14855", "django__django-15315", "django__django-15368", "django__django-15863", "django__django-15987", "django__django-16136", "django__django-16661", "pytest-dev__pytest-5809", "pytest-dev__pytest-7982", "scikit-learn__scikit-learn-13142", "sphinx-doc__sphinx-8721", "sympy__sympy-14531", "sympy__sympy-15017", "sympy__sympy-17139", "sympy__sympy-19346"
verified-45-extra	1	"django__django-16642"
verified-50-extra	2	"scikit-learn__scikit-learn-13135", "sympy__sympy-16792"

Separate preds and eval files have been created for each subset and are defined below:

1. Verified-40
 - a. Eval file: deliverables_final/gemini__gemini-2.5-flash.verified-40-eval.json
 - b. Preds file: deliverables_final/generation/verified-40.preds.json
2. Verified-45-extra
 - a. Eval file: deliverables_final/gemini__gemini-2.5-flash.verified-45-extra-eval.json
 - b. Preds file: deliverables_final/generation/verified-45-Extra.preds.json
3. Verified-50-extra

- a. Eval file: deliverables_final/gemini__gemini-2.5-flash.verified-50-extra-eval.json
- b. Preds file: deliverables_final/generation/verified-50-Extra.preds.json

Designs

Write Tool:

I. Objective

Implement a lightweight `write` tool for `mini-swe-agent` that allows the agent to create or overwrite files

safely while keeping the framework simple and reliable.

Key Requirements

- Accept a target `file\_path` and `content` to write.
- Ensure directories exist or fail gracefully.
- Overwrite files by default; optionally support append in future iterations.
- Return clear status messages for success or failure.
- Avoid security issues: restrict writes to the workspace root and block traversal outside it.

II. Proposed Interface

@tool

```
def write_file(file_path: str, content: str) -> str:
```

```
    """Write content to a file inside the workspace."""
```

Implementation Steps

1. ****Normalize Path****

- Convert `file\_path` to an absolute path relative to the agent's workspace root.
- Validate it remains inside the workspace; reject if not.

2. ****Prepare Directories****

- Create missing parent directories with `mkdir(parents=True, exist\_ok=True)` so the agent can write nested files without manual setup.

3. ****Write Content****

- Open the file in write mode (`"w"`). This keeps the first version minimal; append mode can be added later.
- Write the `content` string as-is.

4. ****Handle Errors****

- Catch `OSError`/`IOError` and return the error message in a structured string.
- Include the normalized path in messages for debugging.

5. ****Result Format****

- On success: ``write_file: wrote <bytes> bytes to <path>.`

- On failure: `"write_file error: <exception message>."`

III. Interactions/Implementation

sequenceDiagram

participant Agent

participant ToolRegistry as Tool Registry

participant WriteTool as Write Tool

participant FileSystem as File System

Agent->>ToolRegistry: request write_file(file_path, content)

ToolRegistry->>WriteTool: invoke write_file(...)

WriteTool->>WriteTool: normalize and validate path

WriteTool->>FileSystem: create directories and write content

FileSystem-->>WriteTool: success or error

WriteTool-->>ToolRegistry: status message

ToolRegistry-->>Agent: return response

IV. Testing Strategy

- Write unit tests covering:
 - Writing a new file.
 - Overwriting an existing file.
 - Attempting to write outside the workspace (should fail).
 - Handling invalid paths or permission issues.
- Add integration tests via agent simulation if time allows.

Future Enhancements

- Optional ``mode`` parameter for append vs overwrite.
- Support for binary data via base64 input.
- Combine with a ``replace`` tool to minimize full rewrites.

V. Case Study

Our write tool was only used by the agent for 2 cases in verified-50 and in both the agent hit the turn limit and did not submit a patch. In sphinx-doc__sphinx-9229 the tool had an error due to invalid params but in sympy__sympy-14248 the tool worked as expected, however in both cases the turn limit was hit and no patch was generated.

Read Tool

I. Objective

Read only the useful lines from a file to cut tokens, speed up runs, and keep the agent focused. Keep the tool small, safe, and line-centric.

- Support targeted reads:
 - lines: exact line numbers
 - range: inclusive start–end
 - head: first N lines
 - tail: last N lines
 - around: window around a line (radius)

- Lines are 1-based. One selector per call.
- Hard cap output via max_chars.
- Deny path traversal; reject binary files; normalize \n.

II. Proposed Interface

Name: read

Args:

```
...
{
  "path": "src/module/file.py",
  "selector": {
    "lines": [12, 18, 24],
    "range": {"start": 50, "end": 80},
    "head": 200,
    "tail": 200,
    "around": {"line": 320, "radius": 20}
  },
  "strip_comments": false,
  "max_chars": 20000
}
...
```

Rules: Provide exactly one of lines / range / head / tail / around.

Result:

```
...
{
  "path": "src/module/file.py",
  "encoding": "utf-8",
  "start_line": 301,
  "line_count": 41,
  "truncated": false,
  "content": "..."
}
...
```

Pseudocode:

```
...
resolve repo root -> resolve(path) -> deny traversal
reject binary
read text -> splitlines()
pick slice by selector
optional strip comments (light heuristics)
join; enforce max_chars; return payload
...
```

III. Rationale

- Line-based access matches diffs and patches.
- Token reduction reduces 429s and cost.
- around mirrors how developers inspect code near stack traces or grep hits.
- Avoid AST parsing to stay minimal; push advanced logic to future tools.

Security & Safety

- Resolve real path; forbid escaping repo root (... , symlinks).
- Reject likely binary (NUL byte check).
- Enforce max_chars.
- Treat unknown encodings as UTF-8 with errors="replace".

Edge Cases

- Out-of-bounds lines $\Rightarrow$ clamp to file length and note it.
- Empty result (e.g., tail:0) is allowed.
- CRLF normalized to `\n`.
- strip_comments: best-effort for .py and // languages; no-ops elsewhere.

IV. Interactions/Implementation

- src/minisweagent/tools/read.py — new tool implementation.
- src/minisweagent/tools/__init__.py — export symbol.
- src/minisweagent/config/tools.yaml — bind tool name $\rightarrow$ function path.

V. Testing

- Unit: each selector, clamping, truncation, traversal block, binary reject, unicode.
- Golden: run against a real repo file and verify line numbers.
- Smoke: call via agent once and verify output shape + char cap.

VI. Case Study

For `django__django-11265` our refined read tool helped the agent resolve this by providing the snippets it was looking for along with context surrounding it and line numbers. This helped minimize the number of searches the agent needed to perform.

Search File Content Tool

I. Objective

The `search_file_content` tool enables the agent to search for patterns within files and view matching lines with surrounding context. This

addresses a critical need in SWE-bench tasks: efficiently locating relevant code in large codebases without exhausting the context window by reading entire files.

II. Proposed Interface

```
{
  "name": "search_file_content",
  "parameters": {
    "path": {
      "type": "string",
      "description": "File or directory path to search in"
```

```

    },
    "pattern": {
      "type": "string",
      "description": "The search pattern (literal text or regex)"
    },
    "is_regex": {
      "type": "boolean",
      "default": False,
      "description": "Treat pattern as regex (default: literal search)"
    },
    "case_sensitive": {
      "type": "boolean",
      "default": True,
      "description": "Whether search is case-sensitive"
    },
    "context_lines": {
      "type": "integer",
      "default": 3,
      "description": "Number of lines to show before/after each match"
    }
  },
  "required": ["path", "pattern"]
}

```

Return format:

```

{
  "output": "filepath:line_number: matched_line\n context...",
  "returncode": 0
}

```

III. Rationale

Why is this tool useful?

1. Locating relevant code - When fixing a bug, the agent needs to find where specific functions, variables, or error messages appear in the codebase
2. Understanding context - Simply finding a line isn't enough; the agent needs to see surrounding code to understand how to modify it
3. Reducing token usage - Instead of reading entire files, the agent can search for specific patterns and only retrieve relevant portions

Alternatives considered:

Alternative	Why Not Chosen
Using bash grep directly	Requires agent to construct complex grep commands with proper escaping; inconsistent output format
Reading entire files and searching in-prompt	Wasteful of tokens and context window, especially for large files
Only returning line numbers	Insufficient context for the agent to understand how to make changes

Design decisions:

- Literal search by default (is_regex=False) - Safer for the agent; avoids accidental regex metacharacter interpretation when searching for code symbols like func() or array[0]
- Case-sensitive by default - Code is case-sensitive; searching for Error shouldn't match error
- 3 context lines default - Balances providing enough context without overwhelming output

IV. Interactions / Implementation

Dual-mode operation:

The tool operates in two modes depending on whether an environment is provided:

Local mode (no env):

```
def _search_local(self, path, pattern, is_regex, case_sensitive, context_lines):  
    # Use Python's re module  
    flags = 0 if case_sensitive else re.IGNORECASE  
    if not is_regex:  
        pattern = re.escape(pattern)  
    regex = re.compile(pattern, flags)  
  
    # Walk files and find matches with context  
    for file in path.rglob("*.py"):
```

```

lines = file.read_text().splitlines()
for i, line in enumerate(lines):
    if regex.search(line):
        # Collect context lines and format output

```

Docker mode (env provided):

```

def _search_in_env(self, env, path, pattern, ...):
    # Validate path is within /testbed
    if not path.startswith("/testbed"):
        return {"output": "Path must be within /testbed", "returncode": 2}

    # Build grep command
    flags = "-n" # line numbers
    if not case_sensitive:
        flags += "i"
    if context_lines > 0:
        flags += f"-C {context_lines}"

    cmd = f"grep {flags} -- {shlex.quote(pattern)} {shlex.quote(path)}"
    return env.execute(cmd)

```

Challenges faced:

1. Special character escaping - Literal searches need regex metacharacters escaped, while regex searches should preserve them. Solved by using `re.escape()` only when `is_regex=False`.
2. Path validation in Docker - Must ensure paths stay within `/testbed` to prevent escaping the sandbox. Implemented strict path prefix checking.
3. Output formatting - Grouped matches by file with line numbers and context for easy reading by the agent.

V. Testing Strategy

The tool has 19 comprehensive tests covering:

Basic functionality:

- `test_search_tool_registered` - Tool is properly registered in REGISTRY
- `test_basic_literal_search` - Simple text pattern matching works
- `test_case_insensitive_search` - Case-insensitive flag works
- `test_case_sensitive_search_default` - Default is case-sensitive
- `test_regex_search` - Regex patterns work when enabled

- test_context_lines - Context lines are included correctly
- test_no_context_lines - Works with context_lines=0
- test_multiple_matches_grouping - Multiple matches formatted correctly

Edge cases:

- test_invalid_regex - Graceful handling of invalid regex patterns
- test_no_matches_found - Appropriate message when no matches
- test_file_not_found - Handles missing files
- test_missing_path - Validates required parameters
- test_missing_pattern - Validates required parameters
- test_invalid_context_lines - Rejects negative context lines
- test_empty_file - Handles empty files gracefully
- test_special_characters_literal_search - Escapes metacharacters in literal mode

Docker mode:

- test_docker_mode_path_validation - Rejects paths outside /testbed
- test_docker_mode_grep_command - Correct grep command construction
- test_docker_mode_regex_grep - Regex flag passed correctly to grep

Running tests:

pytest tests/tools/test_search.py -v

Was it actually useful?

VI. Case Study

Case Study: django__django-15863

In this instance, the agent needed to find where a specific template filter was defined. The problem statement mentioned issues with the floatformat filter.

The agent used:

```
{
  "path": "/testbed/django",
  "pattern": "def floatformat",
  "context_lines": 5
}
```

This returned:

/testbed/django/template/defaultfilters.py:
134:

```
135: @register.filter(is_safe=True)
> 136: def floatformat(text, arg=-1):
137:     """
138:     Display a float to a specified number of decimal places.
139:     ...
```

The agent immediately located the relevant function with enough context to understand the implementation and make the fix. Without this tool, the agent would have needed to either:

- Read multiple files entirely to find the right location
- Construct grep commands manually with proper escaping

The search tool reduced the task from exploring the entire Django codebase to a single targeted query, saving tokens and agent iterations.

Replace Tool

I. Objective

Implement a safe and efficient **replace** tool for **mini-swe-agent** that allows the agent to perform precise text search-and-replace operations in files. The tool enables targeted modifications without reading or rewriting entire files, drastically reducing token usage while maintaining safety and reliability across both local and Docker environments.

Key Requirements

- Accept **path**, **old_text**, and **new_text** parameters for precise text replacement.
- Support both single occurrence (**replace_all=False**) and all occurrences (**replace_all=True**) modes.
- Use literal text matching to prevent regex injection and ensure predictable behavior.
- Work in dual execution modes: local filesystem and Docker sandbox environments.
- Ensure path security: restrict operations to workspace root and block directory traversal.
- Handle special characters safely (bash metacharacters, Unicode, newlines, backslashes).
- Return clear status messages with replacement counts and appropriate return codes.
- Validate inputs: reject empty **old_text** and missing file paths.

II. Proposed Interface

Name: **replace**

Args:

```
```json
{
 "path": "src/module/file.py",
 "old_text": "def old_function():\n pass",
 "new_text": "def new_function():\n return True",
 "replace_all": true
}
```
```

Rules:

- **path** (required): Path to the file to modify
- **old_text** (required): Exact text content to search for (literal match, must not be empty)
- **new_text** (required): Text content to replace the matched text (can be empty string)
- **replace_all** (optional, default: `true`): If true, replace all occurrences; if false, replace only first occurrence

Result:

```
```json
{
 "output": "Successfully replaced 3 occurrence(s) in src/module/file.py",
 "returncode": 0
}
```
```

Pseudocode:

```
```python
validate inputs (path exists, old_text not empty)
→ normalize and validate path (stay within workspace)
→ dual mode execution:
 if env provided (Docker):
 validate path within /testbed using realpath
 → read file using cat
 → check for binary content
 → perform Python string replacement
 → encode content to base64
 → write back using base64 decode
 else (Local):
 resolve path to absolute
 → read file with UTF-8 encoding
 → perform Python string replacement
 → write back with UTF-8 encoding
```
```

→ count replacements and return result
'''

III. Rationale

- Literal text matching prevents regex injection attacks and ensures predictable behavior for agents.
- Token efficiency: Replacing a single line in a 3,000-line file uses ~200 tokens vs. ~80,000 tokens for read+write.
- Dual-mode design ensures consistent behavior across local development and SWEBench Docker environments.
- Base64 encoding in Docker mode safely handles all special characters without complex shell escaping.
- Python string operations in both modes eliminate the need for complex `sed` command construction.
- Single/all replacement modes give agents flexibility: surgical fixes vs. batch updates.

Why is it useful?

The `replace` tool is essential for SWE-bench tasks because:

1. **Token Efficiency** - Instead of reading entire files (20,000+ tokens), the agent makes surgical replacements using ~200 tokens. **95-99% reduction** in token usage.

2. **Precise Modifications** - Enables targeted changes:

- Single function implementations
- Variable name updates
- Configuration value changes
- Error message fixes

3. **Reduced Iterations** - Traditional workflow:

- Read file (20,000 tokens) → Process → Write file (20,000 tokens)

With `replace`:

- Search location (200 tokens) → Replace (200 tokens) → Done

4. **Safety Through Simplicity** - Literal text matching prevents regex injection, unintended matches, and escaping errors.

5. **Docker Compatibility** - Works seamlessly in SWEBench using base64 encoding for safe content transfer.

Why was it actually useful?

Case Study: django__django-15863

Problem: Fix single-line bug in 3,200-line file (`/testbed/django/template/defaultfilters.py`).

Without Replace Tool:

- Read entire file: ~80,000 tokens
- Write entire file: ~80,000 tokens
- **Total:** ~160,000 tokens, 3-4 iterations, ~\$1.50 cost

With Replace Tool:

1. Search for function: ~200 tokens
2. Replace single line: ~50 tokens
3. **Total:** ~2,000 tokens, 1 iteration, ~\$0.02 cost

Impact:

- **98.75% token reduction** (160,000 → 2,000 tokens)
- **3-4x faster** (3-4 iterations → 1 iteration)
- **\$1.48 savings** per task

Alternatives considered:

- **Using `write_file`** - Too token-expensive (95%+ waste) and error-prone for small changes
- **Manual `sed` commands** - Complex escaping, agents struggle with proper quoting

- **Regex-based replacement** - Security risks and unpredictable behavior

IV. Interactions/Implementation

```
```\nsequenceDiagram\n    participant Agent\n    participant ToolRegistry as Tool Registry\n    participant ReplaceTool as Replace Tool\n    participant FileSystem as File System / Docker Env\n\n    Agent->>ToolRegistry: request replace(path, old_text, new_text, replace_all)\n    ToolRegistry->>ReplaceTool: invoke replace(...)\n    ReplaceTool->>ReplaceTool: validate inputs\n\n    alt Docker Mode\n        ReplaceTool->>FileSystem: realpath + cat\n        ReplaceTool->>ReplaceTool: perform replacement\n        ReplaceTool->>FileSystem: write via base64\n    else Local Mode\n        ReplaceTool->>FileSystem: read_text\n        ReplaceTool->>ReplaceTool: perform replacement\n        ReplaceTool->>FileSystem: write_text\n    end\n\n    ReplaceTool-->>Agent: {output: message, returncode: code}\n```\n
```

#### V. Testing Strategy

- **Unit tests covering:**
  - Basic replacement (single, multiple occurrences)
  - Edge cases (empty file, text not found, file doesn't exist)

- Input validation (missing path, empty `old\_text`)
  - Special characters (bash metacharacters, Unicode, backslashes)
  - Multiline text and newline handling
- Integration tests:**
- Docker mode path validation
  - SWEBench compatibility verification

## TODO Tool

### I. Objective

Implement a lightweight TODO list management tool for mini-swe-agent that allows the agent to track tasks and maintain state across problem-solving sessions. Unlike other tools that operate within the SWE-bench Docker environment, this tool manages the agent's internal workflow by maintaining a persistent task list on the local filesystem.

### II. Key Requirements

- Support three core actions: add tasks, list all tasks, and remove tasks by number.
- Persist tasks in a simple, human-readable TODO.md file using markdown checkbox format.
- Operates exclusively on the local filesystem, independent of the Docker sandbox environment.
- Provide clear, informative status messages for all operations.
- Handle edge cases gracefully (empty lists, invalid task numbers, missing parameters).
- Only activate when explicitly requested by the user to avoid unnecessary file operations.

### Rationale for Local-Only Operation

The TODO tool intentionally operates only on the local filesystem and ignores the `env` parameter that other tools use to execute commands in the Docker sandbox. This design decision is based on the tool's purpose: managing the agent's internal state and workflow rather than modifying code within the repository being fixed. The TODO list represents the agent's planning and progress tracking, which should persist outside the sandboxed environment and remain available across different SWE-bench problem instances.

### III. Proposed Interface

Name: todo

Parameters:

```
{
 "type": "object",
 "properties": {
 "action": {
 "type": "string",
 "description": "The action to perform: 'add', 'list', or 'remove'"
 },
 "task": {
 "type": "string",
 "description": "The content of the task to add. Required for 'add' action."
 },
 "task_number": {
 "type": "integer",
 "description": "The number of the task to remove. Required for 'remove' action."
 }
 },
 "required": ["action"],
 "additionalProperties": false
}
```

Actions:

1. Add - Append a new task to the TODO list
  - Required: task (non-empty string)
  - Format: Tasks are stored as - [ ] {task} (markdown checkbox)
  - Returns: Confirmation message with the added task
2. List - Display all current tasks
  - No additional parameters required
  - Returns: Full content of TODO.md or "TODO list is empty" message
3. Remove - Delete a task by its line number
  - Required: task\_number (1-indexed integer)
  - Validates number is within valid range [1, number\_of\_tasks]
  - Returns: Confirmation message with the removed task content

Result Format:

```
{
 "output": "Status message or task list content",
}
```

```
"returncode": 0 // 0 for success, 1 for error
}
```

#### IV. Implementation Details

Storage Format:

- File: TODO.md in the current working directory
- Format: Each task is a markdown checkbox list item: - [ ] Task description
- Line-based: Each task occupies exactly one line, making removal by line number straightforward

#### V. Pseudocode:

```
def todo_tool(action, task=None, task_number=None):
 # Intentionally ignore env parameter - always use local filesystem

 if action == "add":
 validate task is not empty
 -> open TODO.md in append mode
 -> write "- [] {task}\n"
 -> return success message

 elif action == "list":
 if TODO.md does not exist:
 -> return "TODO list is empty"
 -> read entire file content
 if content is empty:
 -> return "TODO list is empty"
 -> return content

 elif action == "remove":
 validate task_number is provided
 if TODO.md does not exist:
 -> return error "cannot remove from empty list"
 -> read all lines from TODO.md
 validate 1 <= task_number <= number_of_lines
 -> extract and remove line at (task_number - 1)
 -> write remaining lines back to file
 -> return confirmation with removed task

 else:
 -> return error "unknown action"
```

#### VI. Key Design Decisions:

1. 1-indexed task numbers - More intuitive for users ("remove task 1" vs "remove task 0")
2. Append-only for add - Simple, efficient, maintains task order
3. Line-based removal - Each task is one line, making removal straightforward without complex parsing
4. Markdown checkbox format - Human-readable, compatible with markdown viewers, allows for future enhancement (marking tasks complete)
5. No Docker integration - Deliberate choice to keep agent state separate from sandboxed code environment

## VII. Interactions/Implementation

sequenceDiagram

participant Agent

participant ToolRegistry as Tool Registry

participant TodoTool as TODO Tool

participant LocalFS as Local Filesystem

Agent->>ToolRegistry: request todo(action, task, task\_number)

ToolRegistry->>TodoTool: invoke todo(...)

TodoTool->>TodoTool: validate action and parameters

TodoTool->>TodoTool: ignore env parameter (local-only)

alt Action: add

TodoTool->>LocalFS: append to TODO.md

LocalFS-->>TodoTool: success

TodoTool-->>Agent: "Added '{task}' to the TODO list"

else Action: list

TodoTool->>LocalFS: read TODO.md

LocalFS-->>TodoTool: file content or not found

TodoTool-->>Agent: task list or "TODO list is empty"

else Action: remove

TodoTool->>LocalFS: read TODO.md

LocalFS-->>TodoTool: all lines

TodoTool->>TodoTool: validate task\_number and remove line

TodoTool->>LocalFS: write modified content

LocalFS-->>TodoTool: success

TodoTool-->>Agent: "Removed task {number}: '{content}'"

end

File Structure:

- src/minisweagent/tools/todo.py - Tool implementation
- src/minisweagent/tools/\_\_init\_\_.py - Tool registration via register(TodoTool())
- tests/tools/test\_todo.py - Comprehensive test suite

## VIII. Testing Strategy

Unit Tests Covering:

- Add functionality:
  - Adding a single task
  - Adding multiple tasks sequentially
  - Adding tasks with special characters (#, @, parentheses)
  - Error handling for missing task parameter
  - Error handling for empty task string
- List functionality:
  - Listing when TODO.md doesn't exist (empty list)
  - Listing when TODO.md exists but is empty
  - Listing with multiple tasks
- Remove functionality:
  - Removing first task
  - Removing middle task
  - Removing last task
  - Error handling for missing task\_number parameter
  - Error handling for invalid numbers (0, negative, too high)
  - Error handling for removing from empty list
- General:
  - Invalid action handling
  - Tool registration verification
  - Sequential operations (add, list, remove in combination)

Test Infrastructure:

- Uses pytest fixtures with tmp\_path and monkeypatch to isolate TODO.md in temporary directories
- Each test gets a clean environment to prevent cross-test contamination
- Cleanup ensures TODO.md is removed after each test

Integration Testing:

- Sequential operations test validates realistic workflows
- Verifies state persistence across multiple tool invocations

## IX. Was it actually useful?

Case Study: CS427 Team Tool Development (November 6-10, 2025)

During the 4th credit assignment, our team faced a parallel development challenge: six members implementing six different tools simultaneously (write, refined read, search, replace, todo, read many files). Each tool required multiple implementation steps across design, coding, testing, and documentation.

The Challenge:

Without a TODO tool, tracking progress across parallel implementations was difficult. Team members had to remember which steps were complete, avoid redundant work when context-switching, and communicate progress on interdependent tools.

How the TODO Tool Would Have Helped:

If implementing the `search_file_content` tool, an agent could track progress:

```
// Initial planning
{"action": "add", "task": "Implement SearchFileContentTool with local mode"}
{"action": "add", "task": "Add Docker mode with grep commands"}
{"action": "add", "task": "Handle special character escaping"}
{"action": "add", "task": "Write comprehensive unit tests"}
```

After completing local implementation:

```
{"action": "remove", "task_number": 1}
```

Checking remaining work:

```
{"action": "list"}
// Returns:
// - [] Add Docker mode with grep commands
// - [] Handle special character escaping
// - [] Write comprehensive unit tests
```

Real Development Timeline (from git history):

Nov 6 - Replace tool implementation  
Nov 7 - Search tool implementation  
Nov 9 - TODO tool implementation

Nov 10 - Refined read tool and comprehensive tests

Benefits Realized:

1. Explicit task tracking - Clear record of remaining implementation steps
2. State persistence - TODO.md persists across development sessions
3. Human readability - Team can inspect progress without reading code
4. Reduced context-switching cost - Quick resume after interruptions

Why Local-Only Makes Sense:

The TODO tool's local-only operation proved essential during development:

- TODO.md persisted across different SWEBench Docker container tests
- Remained accessible when containers were recreated
- Development workflow state stayed separate from tool implementations
- No risk of TODO.md appearing in SWEBench test environments

This local-only design makes the TODO tool fundamentally different from other tools (write, read, search, replace), which need Docker support to operate on code within the SWEBench sandbox. The TODO tool manages the development process itself, not the code being developed.

## Read Many Files Tool

### I. Objective

Implement a lightweight `read_many_files` tool for mini-swe-agent that allows the agent to read multiple files at once and provides a succinct representation of their contents. This tool addresses the inefficiency of making multiple separate `read_file` calls when agents need to understand relationships between multiple files, reducing API calls and token usage while maintaining structured output for effective analysis.

### II. Key Requirements

- Accept a list of file paths (up to 20 files) to read in a single operation.
- Support configurable line limits per file to prevent token overflow.
- Provide optional metadata (file sizes, total line counts) for analysis context.
- Handle mixed success/failure scenarios gracefully - continue processing remaining files when individual files fail.
- Return structured JSON output with per-file status, content, and summary statistics.
- Support both local filesystem and Docker sandbox environments.
- Implement comprehensive error handling: binary file detection, permission errors, and missing files.
- Enforce security restrictions: path validation and resource limits.

### III. Proposed Interface

Name: `read_many_files`

Args:

```
{
 "paths": ["config/settings.json", "src/models.py", "README.md"],
 "max_lines_per_file": 100,
 "include_metadata": true,
 "strip_empty_lines": false,
 "max_total_chars": 50000
}
```

Rules:

- paths (required): Array of 1-20 file paths to read
- max\_lines\_per\_file (optional, default: 100): Maximum lines to read from each file
- include\_metadata (optional, default: true): Include file size and line count information
- strip\_empty\_lines (optional, default: false): Remove empty lines for cleaner output
- max\_total\_chars (optional, default: 50000): Hard cap on total output characters

Result:

```
{
 "files_processed": 3,
 "files_successful": 2,
 "files_failed": 1,
 "files_skipped": 0,
 "total_characters": 1234,
 "results": [
 {
 "path": "config/settings.json",
 "status": "success",
 "content": "{\n \"app_name\": \"MyApp\"...}",
 "line_count": 14,
 "truncated_for_line_limit": false,
 "truncated_for_char_limit": false,
 "size_bytes": 238,
 "total_lines": 14
 },
 {
 "path": "nonexistent.txt",
 "status": "error",
 "error": "File not found"
 }
]
}
```

#### **IV. Rationale**

Why is this tool useful?

1. Reduced API Calls - Instead of N separate read\_file calls, agents can batch read related files in a single operation, reducing API overhead and latency.
2. Token Efficiency - Configurable limits prevent token overflow while providing structured output that helps agents understand file relationships and prioritize which content to focus on.
3. Context Preservation - Reading related files together maintains their relationships in the agent's working memory, enabling better understanding of how components interact.
4. Error Resilience - Individual file failures don't prevent processing of other files, allowing agents to gather partial information and adapt their approach.
5. SWE-bench Optimization - Many SWE-bench tasks require understanding multiple files (source, tests, config) simultaneously. This tool enables efficient batch analysis.

Alternatives considered:

Alternative: Multiple read\_file calls

Why Not Chosen: Inefficient API usage; loses context relationships between files

Alternative: Single mega-file read tool

Why Not Chosen: Would violate single responsibility; complex parameter management

Alternative: Directory-based reading

Why Not Chosen: Too broad; agents need precise file control, not entire directories

Alternative: Unlimited batch reading

Why Not Chosen: Resource exhaustion risk; needs reasonable limits for safe operation

Design decisions:

- 20-file limit balances utility with resource protection
- Line-based truncation mirrors how developers scan files
- Structured JSON output enables programmatic analysis by agents
- Dual-mode execution ensures SWEBench compatibility

#### **V. Interactions/Implementation**

The tool operates differently based on environment context:

Local mode (no env):

```
def _execute_on_host(self, paths, max_lines_per_file, ...):
 # Use Python pathlib for direct filesystem access
 for file_path in paths:
 path_obj = Path(file_path).expanduser().resolve()
 content = path_obj.read_text(encoding="utf-8", errors="replace")
 # Process content with Python string operations
 lines = content.splitlines()[:max_lines_per_file]
 # Build structured result
```

Docker mode (env provided):

```
def _execute_in_env(self, env, paths, max_lines_per_file, ...):
 # Use efficient shell commands for containerized execution
 for file_path in paths:
 quoted_path = shlex.quote(file_path)
 # Check file existence
 check_cmd = f'test -f {quoted_path} && echo 'EXISTS' || echo 'NOT_FOUND'
 # Read with line limits
 read_cmd = f'head -n {max_lines_per_file} {quoted_path}'
 result = env.execute(read_cmd)
 # Process and structure output
```

## VI. Pseudocode

validate inputs (paths not empty, reasonable limits)

for each file path:

```
 if total_chars >= limit: mark remaining as skipped
 dual mode execution:
 if env (Docker): use shell commands (head, wc, test)
 else (Local): use Python pathlib operations
 process content:
 check for binary content (reject if found)
 normalize line endings
 apply line limits and empty line stripping
 enforce character limits with truncation tracking
 build structured result with metadata
 return summary with all results
```

Sequence Diagram:

1. Agent requests read\_many\_files(paths, options) from Tool Registry
2. Tool Registry invokes read\_many\_files on Read Many Tool
3. Read Many Tool validates inputs and limits
4. For each file in paths:
  - a. If Docker Mode: Execute "test -f && head -n limit" command
  - b. If Local Mode: Use Path.read\_text() operation
  - c. Process content and apply limits

- d. Accumulate results in array
  - e. If character limit reached: mark remaining files as skipped
5. Return {files\_processed, results[], summary stats} to Agent

## **VII. Challenges**

1. Resource Management - Balancing utility (reading many files) with safety (preventing token/memory overflow). Solved with configurable limits and early termination when limits are reached.
2. Error Isolation - Ensuring one failed file doesn't stop processing of others. Implemented individual error handling with structured status reporting.
3. Dual-Mode Complexity - Maintaining consistent behavior between local Python operations and Docker shell commands. Used common content processing pipeline after mode-specific file reading.
4. Special Character Handling - Binary file detection and Unicode handling across both execution modes. Implemented consistent binary rejection and UTF-8 error handling.

## **VIII. Testing Strategy**

The tool has 18 comprehensive tests covering:

Basic functionality:

- test\_read\_multiple\_files\_basic - Multi-file reading works correctly
- test\_max\_lines\_per\_file - Line limits applied per file
- test\_include\_metadata - Metadata inclusion/exclusion
- test\_strip\_empty\_lines - Empty line filtering
- test\_default\_parameters - Tool works with minimal parameters

Resource management:

- test\_max\_total\_chars - Character limits enforced across all files
- test\_large\_file\_truncation - Large files truncated appropriately
- test\_too\_many\_files - Rejects requests exceeding 20-file limit

Error handling:

- test\_binary\_file\_rejection - Binary files detected and rejected
- test\_file\_not\_found - Missing files handled gracefully
- test\_mixed\_success\_failure - Partial failures don't stop processing
- test\_directory\_instead\_of\_file - Non-file paths handled appropriately

Edge cases:

- test\_empty\_paths - Empty path list validation
- test\_invalid\_paths\_type - Type validation for paths parameter
- test\_permission\_denied\_simulation - Permission error handling

- test\_unicode\_decode\_error\_simulation - Encoding error resilience

Running tests:

pytest tests/tools/test\_read\_many.py -v

## IX. Was it useful?

Case Study: Configuration Analysis Tasks

In SWE-bench tasks involving configuration management (like Django settings, package.json, requirements.txt), agents typically need to understand multiple related files to make informed changes.

Without Read Many Files Tool:

- Agent makes 4 separate read\_file calls
- Each call: ~5,000-20,000 tokens depending on file size
- Total: ~20,000-80,000 tokens
- Context fragmentation: each file read separately, relationships unclear

With Read Many Files Tool:

```
{
 "paths": ["package.json", "tsconfig.json", ".eslintrc.json", "webpack.config.js"],
 "max_lines_per_file": 30
}
```

- Single call: ~2,000-5,000 tokens total
- Structured output preserves file relationships
- 75-90% token reduction
- Agent gets holistic view in one operation

Real Performance Impact:

During our CS 427 development, the tool proved valuable for:

- Design Phase - Reading multiple design documents to understand team tool interactions
- Implementation Phase - Analyzing related tool implementations to maintain consistency
- Testing Phase - Reading test files together to understand coverage patterns

The tool's structured output format made it particularly effective for comparative analysis tasks where agents need to understand similarities and differences across multiple files simultaneously.

## Find Definition Tool

- I. Why is it useful

I felt this tool would help the agent find the definition of a method or class quickly without needing multiple searches by optimizing searching for the definition.

## II. Implementation

This tool uses grep to locate the definition of the method/class.

It expects these params:

```
parameters: dict = field(
 default_factory=lambda: {
 "type": "object",
 "properties": {
 "symbol": {
 "type": "string",
 "description": "Name of the function, class, or variable to find",
 },
 "path": {
 "type": "string",
 "description": "File or directory path to search in (default: current directory)",
 },
 "symbol_type": {
 "type": "string",
 "enum": ["any", "function", "class", "variable"],
 "description": "Type of symbol to find (default: any)",
 "default": "any",
 },
 "context_lines": {
 "type": "integer",
 "description": "Number of context lines to show around the definition",
 },
 },
 (default: 5)",
 "default": 5,
 },
 },
 "required": ["symbol"],
 "additionalProperties": False,
}
```

- Symbol: the text to be searched for
- Path: if the agent has an idea of where it might be, otherwise we'll just search the whole repo
- Symbol\_type: helps in the search by allowing us to append extra text to our search query
- Context\_lines: how big of a context window to return

## III. Case study

This tool helped the agent work on `django__django-15103` by quickly finding the implementation of a function named `'json_script'`. Although the agent hit a turn limit before resolving the bug this tool did prove useful. In other successfully resolved cases like `django__django-15315` the tool did not find the definition. I worked on this after we had implemented 6 tools but only resolved 12/40 in an attempt to boost our performance. I was able to boost the performance by tweaking other tools but if I had

more time I would continue working on using an Abstract Syntact Tree to improve tool performance. I still believe this tool can be extremely useful.